

Laboratorio de Sistemas Móviles y Ubicuos

Grado en Ingeniería Telemática

DataBinding & ViewModel



Universidad
Rey Juan Carlos

DataBinding

findViewById

Para programar la lógica relacionada con la interfaz de nuestra aplicación necesitamos crear variables y asociarlas a su View mediante *findViewById*. Sin embargo, las IDs son globales a todas las pantallas, por lo que pueden aparecer errores de ejecución si se referencia a una vista que no está en el layout actual.

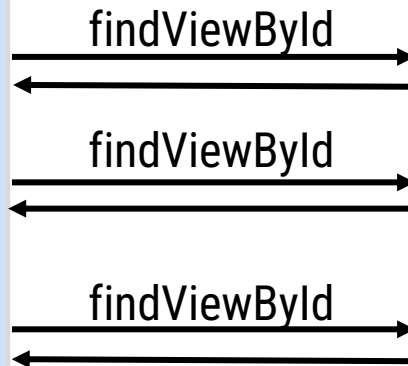
MainActivity.kt

```
val name = findViewById(...)
val age = findViewById(...)
val loc = findViewById(...)

name.text = ...
age.text = ...
loc.text = ...
```

activity_main.xml

```
<ConstraintLayout ... >
  <TextView
    android:id="@+id/name"/>
  <TextView
    android:id="@+id/age"/>
  <TextView
    android:id="@+id/loc"/>
</ConstraintLayout>
```



DataBinding

Como alternativa, se puede utilizar el enfoque *DataBinding* (vinculación de datos), que compila clases que enlazan directamente con las vistas del diseño actual en lugar de tener que asociar a cada vista con una variable manualmente.

MainActivity.kt

```
Val binding:ActivityMainBinding
```

```
binding.name.text = ...  
binding.age.text = ...  
binding.loc.text = ...
```

initialize binding



activity_main.xml

```
<layout>  
  <ConstraintLayout ... >  
    <TextView  
      android:id="@+id/name"/>  
    <TextView  
      android:id="@+id/age"/>  
    <TextView  
      android:id="@+id/loc"/>  
  </ConstraintLayout>  
</layout>
```

Modificar el archivo de compilación

Para utilizar *DataBinding* tenemos que modificar el archivo *build.gradle.kts* y añadir el siguiente bloque de código, que le indica al compilador que se generen las clases necesarias que nos permitan trabajar con los layouts definidos en XML.

```
android {  
    ...  
    buildTypes {  
        ...  
    }  
    buildFeatures{  
        dataBinding = true  
    }  
    ...  
}
```

build.gradle.kts (:app)

Con cada cambio en el Gradle, recuerda sincronizar el proyecto (Sync Now)

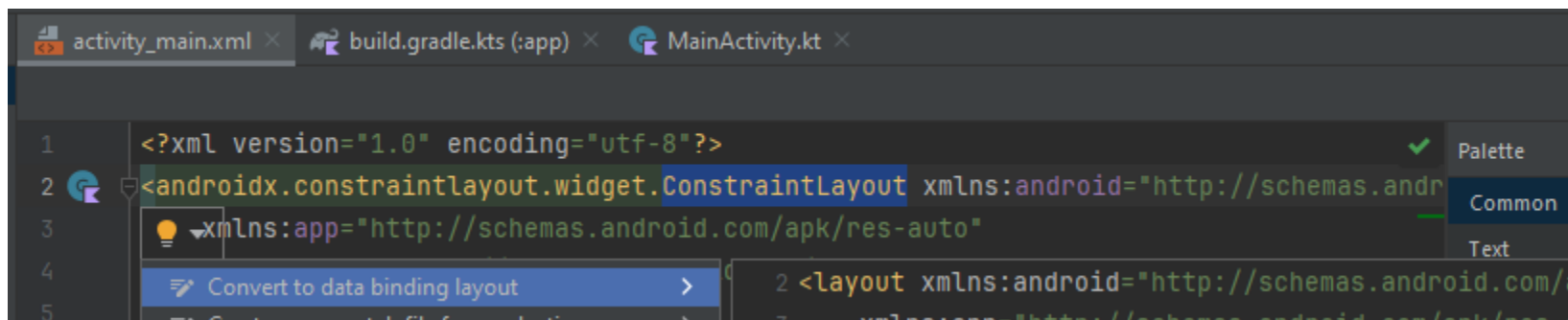
Añadir etiqueta *layout*

Para indicar al compilador qué diseños queremos utilizar con DataBinding, en los XML correspondientes añadimos una etiqueta *layout* que los contenga. La clase generada contendrá referencias a todas las vistas que tengan un ID asignado

```
<layout ... >
  <androidx.constraintlayout.widget.ConstraintLayout>
    <TextView android:id="@+id/usuario" ... />
    <EditText android:id="@+id/contraseña"... />
  </androidx.constraintlayout.widget.ConstraintLayout>
</layout>
```

activity_main.xml

Se puede hacer desde las Context Actions del elemento raíz de nuestro diseño. 



Crear la interfaz con DataBinding en la Actividad

En lugar de llamar a `setContentView()` desde la actividad, llamamos a `DataBindingUtil.setContentView()` y le pasamos el Activity (*this*) y el ID del recurso de layout deseado.

```
class MainActivity : AppCompatActivity() {  
    lateinit var binding: ActivityMainBinding  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        // setContentView(R.layout.activity_main)  
        binding = DataBindingUtil.setContentView(this, R.layout.activity_main)  
  
        binding.usuario.text = "Jorge"  
    }  
}
```

MainActivity.kt

Declarar variables directamente en el *layout*

Se pueden crear variables en el propio layout y hacer que las vistas referencien a su valor, simplificando la lógica en la Actividad.

```
<layout>
  <data>
    <variable name="nombre" type="String" />
  </data>
  ...
  <TextView
    android:id="@+id/usuario"
    android:text="@{nombre}" />
  ...
</layout>
```

activity_main.xml

También sirven algunas expresiones simples
como: "@{nombre.toUpperCase()}"

```
override fun onCreate(savedInstanceState: Bundle?) {
  ...
  binding.nombre= "Jorge" // En lugar de: binding.usuario.text = "Jorge"
}
```

MainActivity.kt

Ejercicio 1

Crea una aplicación para llevar la puntuación de un partido de baloncesto utilizando DataBinding.

La aplicación debe tener un diseño como el que se muestra a la derecha, con una columna para cada equipo y un botón para resetear.

En la columna de cada equipo se mostrará el nombre del equipo, su marcador actual y tres botones que incrementarán el marcador cuando se anote un tiro libre, un tiro de campo o un triple.

¿Qué pasa cuando giramos la orientación del dispositivo?

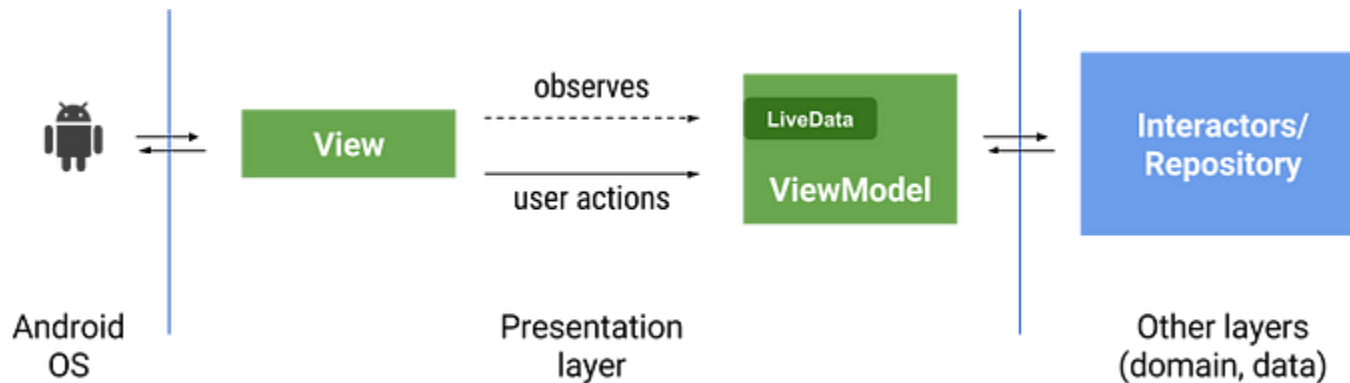


ViewModel

Model - View - View Model (MVVM)

MVVM es un patrón de diseño que tiene por finalidad separar la parte de la interfaz del usuario (View) de la parte de la lógica del negocio (Model), logrando así que la parte visual sea totalmente independiente.

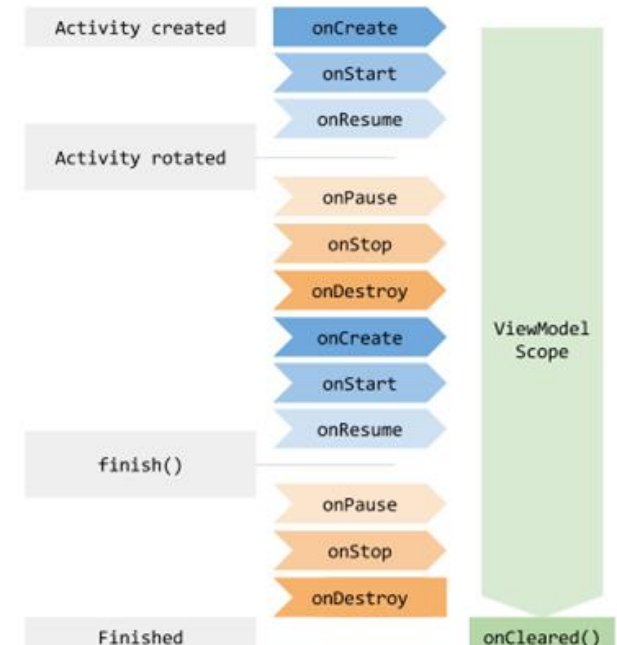
El ViewModel es el tercer componente y se encarga de interactuar como puente entre la Vista y el Modelo.



ViewModel

Un ViewModel permite separar la propiedad de los datos de la lógica del controlador de la interfaz de usuario:

- Prepara los datos para la interfaz de usuario
- No debe hacer referencia a actividades, fragmentos o vistas en la jerarquía de vistas
- Asociado a un ciclo de vida (que tienen la actividad y el fragmento)
- Permite que los datos sobrevivan a los cambios de configuración
- Sobrevive mientras el ámbito está vivo



Añadir dependencias de ViewModel

Para habilitar ViewModel en nuestras aplicaciones, necesitamos añadir algunas dependencias al archivo Gradle de nuestro módulo.

build.gradle.kts (:app)

```
dependencies {  
    val activity_version = "1.8.2"  
    val lifecycle_version = "2.7.0"  
    ...  
    implementation("androidx.lifecycle:lifecycle-viewmodel-ktx:$lifecycle_version")  
    implementation("androidx.activity:activity-ktx:$activity_version")  
    ...  
}
```

Crear el ViewModel

Para crear un ViewModel hay que crear una clase que extienda la clase abstracta ViewModel.

```
class LoginViewModel: ViewModel() {  
    var nombre: String = "Jorge"  
    var contrasena: String = ""  
  
    fun cambiarNombre(nuevo: String){  
        nombre = nuevo  
    }  
}
```

LoginViewModel

abstract class ViewModel

Public constructors

<init>()

ViewModel is a class that is responsible for preparing and managing the data for an [Activity](#) or a [Fragment](#).

Protected methods

open [Unit](#)

[onCleared\(\)](#)

Usar el ViewModel en la Actividad

En la Actividad, instanciamos el ViewModel y modificamos la lógica para que las vistas que muestran la información hagan referencia a los atributos de la clase ViewModel.

Cualquier cambio en los datos se hace a través de llamadas a funciones del ViewModel.

```
class MainActivity : AppCompatActivity() {  
    val viewModel: LoginViewModel by viewModels()  
    override fun onCreate(savedInstanceState: Bundle?) {..  
        // binding.usuario.text = "Jorge"  
        binding.nombre = viewModel.nombre  
  
        binding.button.setOnClickListener{  
            //binding.nombre = "Jonas"  
            viewModel.cambiarNombre("Jonas")  
        }  
    }  
}
```

MainActivity.kt

Ejercicio 2

Siguiendo con el ejercicio anterior:

1. Crea un ViewModel que permita:
 1. Almacenar el estado de los marcadores de ambos equipos.
 2. Incrementar la puntuación de cada equipo en 1, 2 o 3 puntos.
 3. Resetear los marcadores.
2. Adapta la MainActivity para que haga uso del ViewModel implementado.

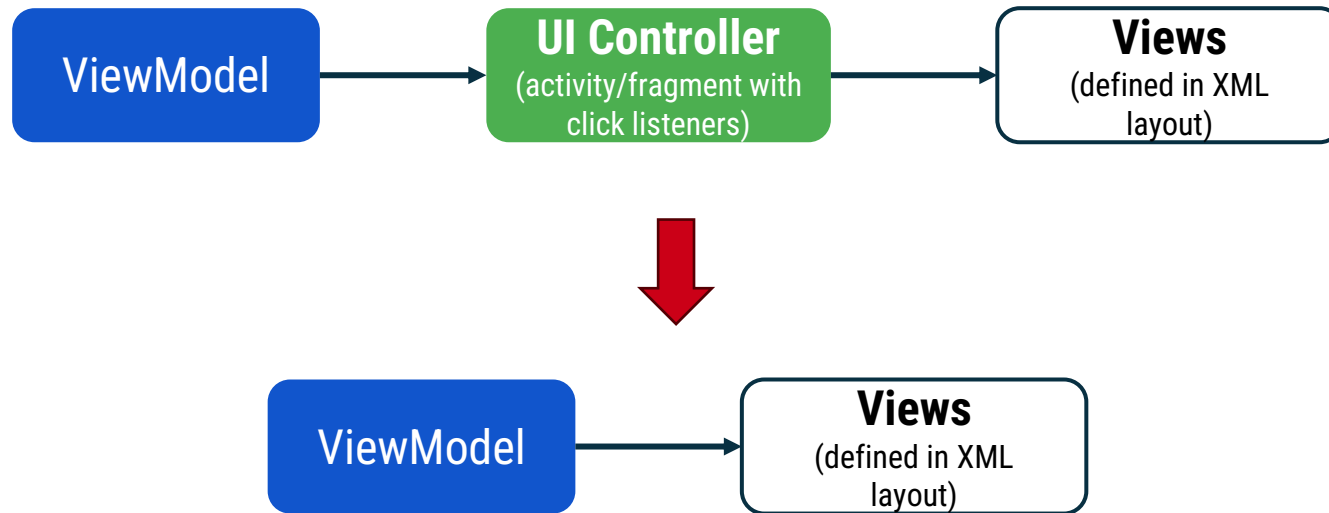
¿Qué pasa cuando pulsamos en los botones?

¿Qué pasa cuando giramos la orientación del dispositivo?

Conectando el XML con el ViewModel

Los objetos ViewModel contienen los datos de la aplicación. En el diseño, sería más sencillo si la instancia ViewModel se comunicara directamente con las vistas, sin depender de los controladores de interfaz de usuario como intermediarios.

Al pasar el ViewModel al DataBinding, se pueden automatizar parte de la comunicación entre las vistas y los objetos ViewModel.



Viewmodel con DataBinding

Para utilizar el ViewModel con DataBinding, se debe declarar una variable en el layout con el tipo correspondiente y asignar el viewModel a dicha variable en la actividad.

```
<layout>
  <data>
    <variable name="viewModel" type="urjc.lsmu.viewmodelsimple.LoginViewModel" />
  </data>
  ...
```

activity_main.xml

```
class MainActivity : AppCompatActivity() {
  lateinit var binding: ActivityMainBinding
  val viewModel: LoginViewModel by viewModels()
  override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = DataBindingUtil.setContentView(this, R.layout.activity_main)
    binding.viewModel = viewModel
    ...
  }
}
```

MainActivity.kt

Actualizando la UI con el ViewModel

De esta forma, las vistas podrán hacer referencia a los atributos del ViewModel.

```
<TextView
    android:id="@+id/usuario"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@{viewModel.nombre}" />
```

actviity_main.xml

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    binding = DataBindingUtil.setContentView(this, R.layout.activity_main)
    binding.viewModel = viewModel
    // binding.usuario.text = viewModel.nombre
    binding.button.setOnClickListener{
        viewModel.cambiarNombre("Laura")
        binding.usuario.text = viewModel.nombre
    }
    ...
}
```

MainActivity.kt

Seguimos necesitando
actualizar el Textview
con cada cambio

Ejercicio 3

Siguiendo con el ejercicio anterior, modifica la aplicación para que haga uso del ViewModel a través de DataBinding.

Referencias

- *Curso Android Development with Kotlin*
- <https://developer.android.com/>